

MagicCode Super-Resolution Software Specification

Document Version: v2.1.0. Release Date: September 4, 2026.

1. Document Overview

This specification defines capabilities, platforms, and functional boundaries of MagicCode Super-Resolution Software v2.1.0. It covers spatial and temporal GPU paths and spatial CPU paths.

1.4 Document Status

Version	Date	Status	Change
v2.1.0	September 4, 2026	Updated	Unified v2.1.0 3-function public ABI, single-library delivery, temporal modes.

2. System Requirements

Platform	GPU backends	Library	Temporal
Android	Vulkan, OpenGL ES	lib/android/libmagic_sr.a	Temporal GPU yes; CPU temporal no
iOS / iPadOS	Metal	lib/ios/libmagic_sr.a	Temporal GPU yes; CPU temporal no
macOS (Apple Silicon)	Metal	lib/mac_arm/libmagic_sr.a	Temporal GPU yes; CPU temporal no
macOS (x86_64)	CPU	lib/mac_x86/libmagic_sr.a	Spatial AVX2; Temporal no
Windows	Vulkan, Direct3D 11, OpenGL 4.3	lib/windows/libmagic_sr.lib	Temporal GPU yes; CPU temporal no

- Input size [64, 4032]. Spatial scale [1, 8]. Temporal scale (1, 8].
- CPU spatial: x86_64 AVX2 and ARM64 NEON buffer paths. CPU temporal is not available.

3. Functional Capabilities

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

Spatial modes upscale a single frame. Temporal modes accumulate history and require depth, motion, jitter, and camera parameters through `magic_frame_t` / `temporal_frame_t`.

Temporal Super-Resolution

v2.1.0 provides temporal super-resolution through the unified public C ABI: `libmagic_sr.a` (Windows: `libmagic_sr.lib`) + `interface/mc_interface.h` + `MC_Enable` / `MC_Disable`. A single function, `MC_Enable`, handles implicit handle initialization, dynamic reconfiguration, temporal execution, and execution status query.

Modes

Value	Enumerator	Role
0	<code>SPATIAL_SPEED_MODE</code>	Spatial, throughput first
1	<code>SPATIAL_BALANCED_MODE</code>	Spatial, quality/cost trade-off
2	<code>TEMPORAL_SPEED_MODE</code>	Temporal, throughput first
3	<code>TEMPORAL_BALANCED_MODE</code>	Temporal, canonical FSR-family path

- Temporal `scaler_factor` is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- `TEMPORAL_SPEED_MODE` on Metal, Vulkan, and OpenGLS currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- `TEMPORAL_BALANCED_MODE` uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced (Windows Vulkan / Direct3D 11 / desktop GL; Apple Metal) can run edge-resolve; if `enable_sharpening` is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- `magic_frame_t.image_in` / `image_out`: caller-owned color and output GPU resources.
- `magic_frame_t.frame`: non-NULL `temporal_frame_t*` with `struct_size = sizeof(temporal_frame_t)`.
- `temporal_frame_t.depth` and motion at input resolution.
- `jitter_offset_x/y` in input pixels, top-left, +X right, +Y down.
- `frame_index` / `reset_history`; `camera_near` / `camera_far` / `camera_fov_y` (radians).
- `gpu_context` at initial `MC_Enable`: Vulkan requires `physical_device + device`; D3D11 requires `device`; Metal `device` may be NULL (system default); GL/GLES use the current context (library never makes current).
- Vulkan temporal requires `magic_frame_t.command_buffer` to be a recording `VkCommandBuffer`. Metal `command_buffer` is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage `depth_reversed` / `depth_infinite` / `hdr_color` on `input_param_t` (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- `motion_vector_scale_x/y` is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: `MAGICSR_TAAU_AUTO_MASK=0` and `MAGICSR_TAAU_HIST_REJECT=0` can turn those defaults off. They are not a customer configuration path.

4. Resource and synchronization contract

- Caller owns application GPU objects. The library owns internal history, scratch, and pipelines only.
- Vulkan: library does not begin/end/submit/wait. Restore of input layouts after compute; output remains GENERAL.
- Complete prior GPU work before `MC_Disable` or a size-changing reconfiguration.

5. Conformance notes

API support is not the same as device-run verification. Exact 2× has optimized temporal kernels; other rates in (1, 8] are supported by the API. Do not claim every scale or GPU vendor is verified.